

Question-2.10.51

EE25BTECH11022 - sankeerthan

Question

If $\mathbf{a}, \mathbf{b}, \mathbf{c}$ are three non-zero, non-coplanar vectors and

$$\mathbf{b}_1 = \mathbf{b} - \frac{\mathbf{b} \cdot \mathbf{a}}{|\mathbf{a}|^2} \mathbf{a}, \quad \mathbf{b}_2 = \mathbf{b} + \frac{\mathbf{b} \cdot \mathbf{a}}{|\mathbf{a}|^2} \mathbf{a},$$

$$\mathbf{c}_1 = \mathbf{c} - \frac{\mathbf{c} \cdot \mathbf{a}}{|\mathbf{a}|^2} \mathbf{a} + \frac{\mathbf{b} \cdot \mathbf{c}}{|\mathbf{c}|^2} \mathbf{b}_1, \quad \mathbf{c}_2 = \mathbf{c} - \frac{\mathbf{c} \cdot \mathbf{a}}{|\mathbf{a}|^2} \mathbf{a} + \frac{\mathbf{b}_1 \cdot \mathbf{c}}{|\mathbf{b}_1|^2} \mathbf{b}_1,$$

$$\mathbf{c}_3 = \mathbf{c} - \frac{\mathbf{c} \cdot \mathbf{a}}{|\mathbf{c}|^2} \mathbf{a} + \frac{\mathbf{b} \cdot \mathbf{c}}{|\mathbf{c}|^2} \mathbf{b}_1, \quad \mathbf{c}_4 = \mathbf{c} - \frac{\mathbf{c} \cdot \mathbf{a}}{|\mathbf{c}|^2} \mathbf{a} = \frac{\mathbf{b} \cdot \mathbf{c}}{|\mathbf{b}|^2} \mathbf{b}_1,$$

then the set of orthogonal vectors is

- ① $(\mathbf{a}, \mathbf{b}_1, \mathbf{c}_3)$
- ② $(\mathbf{a}, \mathbf{b}_1, \mathbf{c}_2)$
- ③ $(\mathbf{a}, \mathbf{b}_1, \mathbf{c}_1)$
- ④ $(\mathbf{a}, \mathbf{b}_2, \mathbf{c}_2)$

Solution

Let $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$ be three linearly independent (non-coplanar) vectors in $\mathbb{R}^3$. We form a 3×3 matrix whose columns are these vectors:

$$A = \begin{pmatrix} | & | & | \\ \mathbf{a} & \mathbf{b} & \mathbf{c} \\ | & | & | \end{pmatrix} \quad (1)$$

To obtain an orthogonal set, apply the projection formula (Gram-Schmidt): Let $\mathbf{v}_1 = \mathbf{a}$. Orthogonalize $\mathbf{b}$ against $\mathbf{a}$:

$$\mathbf{v}_2 = \mathbf{b}_1 = \mathbf{b} - \text{proj}_{\mathbf{a}} \mathbf{b} = \mathbf{b} - \frac{\mathbf{a}^T \mathbf{b}}{\mathbf{a}^T \mathbf{a}} \mathbf{a} \quad (2)$$

Orthogonalize $\mathbf{c}$ against $\mathbf{a}$ and $\mathbf{b}_1$:

$$\mathbf{v}_3 = \mathbf{c}_1 = \mathbf{c} - \frac{\mathbf{a}^T \mathbf{c}}{\mathbf{a}^T \mathbf{a}} \mathbf{a} - \frac{\mathbf{b}_1^T \mathbf{c}}{\mathbf{b}_1^T \mathbf{b}_1} \mathbf{b}_1 \quad (3)$$

Thus, the matrix with orthogonal columns is:

$$Q = \begin{pmatrix} | & | & | \\ \mathbf{a} & \mathbf{b}_1 & \mathbf{c}_1 \\ | & | & | \end{pmatrix} \quad (4)$$

The columns in Q form an orthogonal set. The set of orthogonal vectors is: $(\mathbf{a}, \mathbf{b}_1, \mathbf{c}_1)$

C-Code

```
#include <stdio.h>
#include <math.h>

void dot_product(double *a, double *b, int dim, double *result) {
    *result = 0.0;
    for (int i = 0; i < dim; ++i) {
        *result += a[i] * b[i];
    }
}

void scalar_mult(double *a, double k, int dim, double *result) {
    for (int i = 0; i < dim; ++i) {
        result[i] = k * a[i];
    }
}

void vector_sub(double *a, double *b, int dim, double *result) {
    for (int i = 0; i < dim; ++i) {
        result[i] = a[i] - b[i];
    }
}
```

```
void gram_schmidt(double *a, double *b, double *c, double *b1,
    double *c3) {
    int dim = 3;
    double a_dot_b, a_dot_a, a_dot_c, c_dot_b1, b1_dot_b1;
    double proj_a_on_b[3], proj_a_on_c[3], proj_b1_on_c[3];

    dot_product(a, a, dim, &a_dot_a);
    dot_product(b, a, dim, &a_dot_b);
    scalar_mult(a, a_dot_b / a_dot_a, dim, proj_a_on_b);
    vector_sub(b, proj_a_on_b, dim, b1);

    dot_product(c, a, dim, &a_dot_c);
    scalar_mult(a, a_dot_c / a_dot_a, dim, proj_a_on_c);
    double temp[3];
    vector_sub(c, proj_a_on_c, dim, temp);

    dot_product(b1, b1, dim, &b1_dot_b1);
    dot_product(c, b1, dim, &c_dot_b1);
```

```
scalar_mult(b1, c_dot_b1 / b1_dot_b1, dim, proj_b1_on_c);

for (int i = 0; i < dim; ++i)
    c3[i] = temp[i] - proj_b1_on_c[i];}
__attribute__((visibility("default")))
void get_orthogonal_vectors(double *a, double *b, double *c,
    double *out_a, double *out_b1, double *out_c3) {
    for(int i=0; i<3; ++i) out_a[i]=a[i];
    gram_schmidt(a, b, c, out_b1, out_c3);
```

```
# Code by GVV Sharma  
# Modified for Problem Solution  
# Released under GNU GPL  
# Calculating area enclosed between curves  
# call.py  
  
import ctypes  
import numpy as np  
  
lib = ctypes.CDLL('./code.so')
```

```
ib.get_orthogonal_vectors.argtypes = [  
    ctypes.POINTER(ctypes.c_double),  
    ctypes.POINTER(ctypes.c_double),  
    ctypes.POINTER(ctypes.c_double),  
    ctypes.POINTER(ctypes.c_double),  
    ctypes.POINTER(ctypes.c_double),  
    ctypes.POINTER(ctypes.c_double)  
]
```

```
def get_orthogonal_vectors(a, b, c):  
    a = np.array(a, dtype=np.float64)  
    b = np.array(b, dtype=np.float64)  
    c = np.array(c, dtype=np.float64)
```

```
out_a = np.zeros(3, dtype=np.float64)
out_b1 = np.zeros(3, dtype=np.float64)
out_c3 = np.zeros(3, dtype=np.float64)
lib.get_orthogonal_vectors(
    a.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
    b.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
    c.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
    out_a.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
    out_b1.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
    out_c3.ctypes.data_as(ctypes.POINTER(ctypes.c_double))
)
return out_a, out_b1, out_c3
```